

گزارش پروژه سوم درس نرم افزار ریاضی

نویسنده : مهرداد زیدی دوزنده

شماره دانشجویی: ۴۰۱۱۳۴۱۷

ادرس مخزن گیت هاب:

<https://github.com/mhz171/MathSoft-P03.git>

این پروژه از سه سوال مجزا تشکیل شده است.

سوال ۱: درونیابی لاگرانژ و اسپلین

در این بخش، نقاط داده $(۲,۳)$ ، $(۳,۵)$ ، $(۴,۸)$ ، $(۵,۵)$ ، $(۶,۲)$ و $(۱,۱)$ در نظر گرفته شده‌اند. هدف این است که با استفاده از درونیابی چندجمله‌ای لاگرانژ و اسپلین مکعبی، ضابطه دقیق درونیابی به دست آید.

۱-۱ روش پیاده‌سازی

ابتدا نقاط داده در آرایه‌های NumPy ذخیره شدند. برای محاسبه لاگرانژ از تابع lagrange در کتابخانه SciPy و برای اسپلین از تابع CubicSpline استفاده شده است. همچنین دقت شد که برای پیاده‌سازی اسپلین، مقادیر x حتماً به صورت صعودی مرتب شوند.

۱-۲ کد پایتون

```
import numpy as np
from scipy.interpolate import CubicSpline, lagrange

x_data = np.array([6, 5, 4, 3, 2, 1], dtype=float)
y_data = np.array([2, 5, 8, 5, 3, 1], dtype=float)

poly_lagrange = lagrange(x_data, y_data)

sort_idx = np.argsort(x_data)
cs = CubicSpline(x_data[sort_idx], y_data[sort_idx])
```

۱-۳ ضابطه لاگرانژ

ضرایب چندجمله‌ای لاگرانژ (از درجه بالا به پایین) به شرح زیر است:

[0.175, -2.95833333, 18.375, -52.04166667, 68.45, -31.]

بر این اساس، ضابطه نهایی به این فرم است:

$$L(x) = 0.175x^5 - 2.958x^4 + 18.375x^3 - 52.042x^2 + 68.45x - 31$$

```
Exact Lagrange interpolating polynomial:
      5      4      3      2
0.175 x - 2.958 x + 18.37 x - 52.04 x + 68.45 x - 31

Polynomial coefficients (highest degree first):
[ 0.175 -2.95833333 18.375 -52.04166667 68.45
-31. ]
```

شکل 1

۴-۱ ضابطه اسپلین مکعبی

معادلات به‌دست‌آمده برای هر بازه عبارت‌اند از:

- بازه [1, 2]: $S_0(x) = 0.666667x^3 - 2.000000x^2 + 3.333333x + 1.000000$
- بازه [2, 3]: $S_1(x) = 0.666667x^3 + 0.000000x^2 + 1.333333x + 3.000000$
- بازه [3, 4]: $S_2(x) = -2.333333x^3 + 2.000000x^2 + 3.333333x + 5.000000$
- بازه [4, 5]: $S_3(x) = 1.666667x^3 - 5.000000x^2 + 0.333333x + 8.000000$
- بازه [5, 6]: $S_4(x) = 1.666667x^3 + 0.000000x^2 - 4.666667x + 5.000000$

```
Exact cubic spline (piecewise polynomials):
S(x) =

On [1, 2]:
  S_0(x) = 0.666667·x³ + -2.000000·x² + 3.333333·x + 1.000000

On [2, 3]:
  S_1(x) = 0.666667·x³ + 0.000000·x² + 1.333333·x + 3.000000

On [3, 4]:
  S_2(x) = -2.333333·x³ + 2.000000·x² + 3.333333·x + 5.000000

On [4, 5]:
  S_3(x) = 1.666667·x³ + -5.000000·x² + 0.333333·x + 8.000000

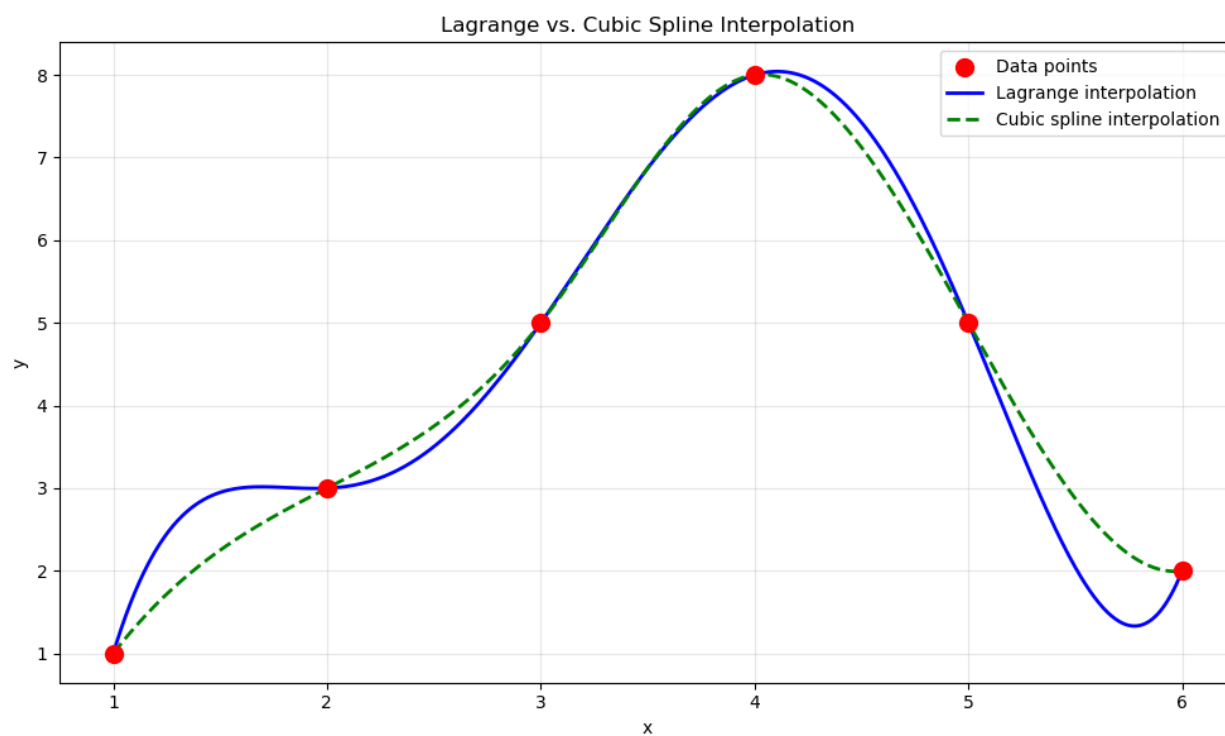
On [5, 6]:
  S_4(x) = 1.666667·x³ + 0.000000·x² + -4.666667·x + 5.000000
```

شکل 2

۵-۱ تحلیل و نتیجه‌گیری

- چندجمله‌ای لاگرانژ یک چندجمله‌ای درجه ۵ است که دقیقاً از تمامی ۶ نقطه مشخص‌شده عبور می‌کند.

- اسپلین مکعبی از ۵ چندجمله‌ای مکعبی تکه‌ای تشکیل شده است که در گره‌های داخلی دارای پیوستگی در مشتق اول و دوم هستند.
- هر دو روش در نقاط داده مقدار یکسانی دارند، اما رفتار آن‌ها در فواصل بین نقاط متفاوت است. لاگرانژ ممکن است نوسانات بیشتری در انتهاها نشان دهد، در حالی که اسپلین رفتار هموارتر و پایدارتری دارد.



شکل 3

سوال ۲: تحلیل زمان اجرای الگوریتم‌ها

۱-۲ روش پیاده‌سازی

فایل داده‌ها با نام s02.xlsx توسط کتابخانه pandas فراخوانی و خوانده شد. نام ستون‌ها به صورت پویا استخراج شدند تا نیازی به ورود دستی داده‌ها به کد نباشد. سپس نمودارهای مختلف (میله‌ای، خطی و جعبه‌ای) برای تحلیل دقیق‌تر رسم شدند.

۲-۲ کد خواندن فایل اکسل

```
import pandas as pd
from pathlib import Path

excel_path = Path('s02.xlsx')
df = pd.read_excel(excel_path)
df.columns = df.columns.str.strip()

size_col = [c for c in df.columns if 'data size' in c.lower()][0]
alg_cols = [c for c in df.columns if c.startswith('Alg.')]

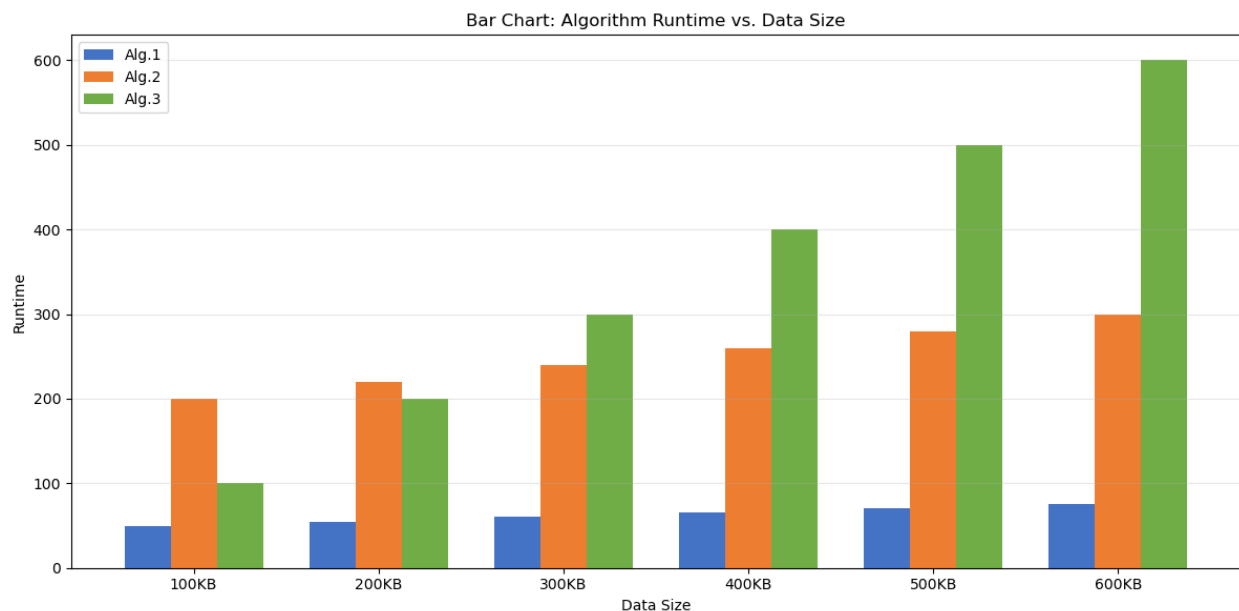
```

۳-۲ جدول داده‌های اولیه (۱۰۰ تا ۶۰۰ کیلوبایت)

سایز داده (Size_KB)	Alg.1	Alg.2	Alg.3
100KB	50	200	100
200KB	55	220	200
300KB	60	240	300
400KB	65	260	400
500KB	70	280	500
600KB	75	300	600

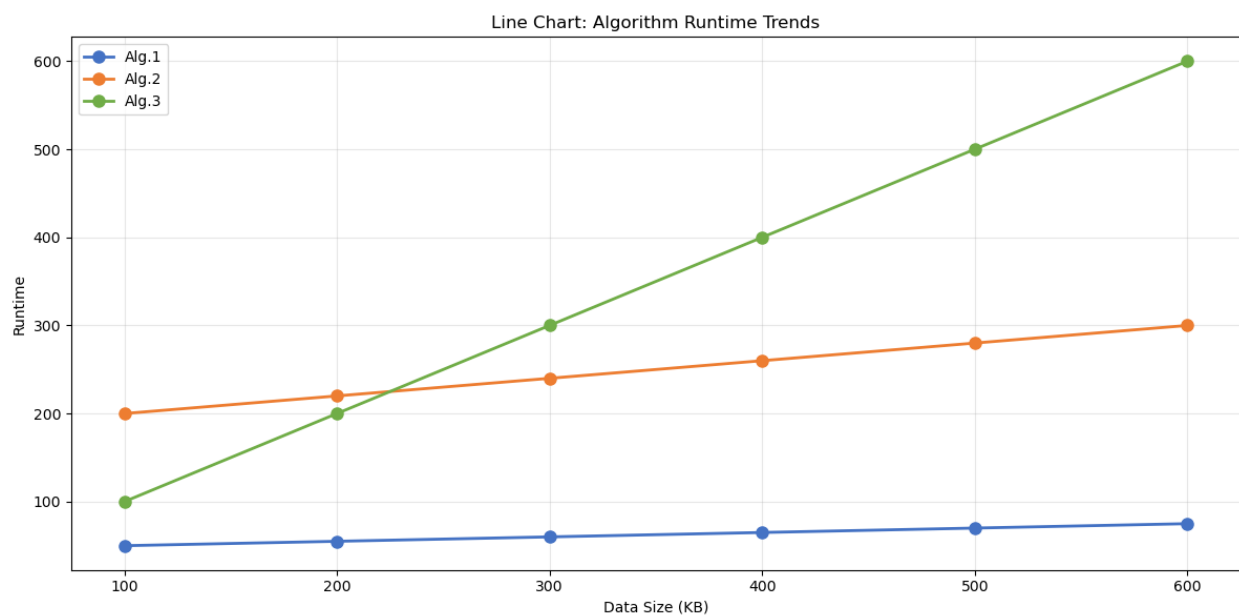
جدول 1

۴-۲ تحلیل نمودارهای اولیه



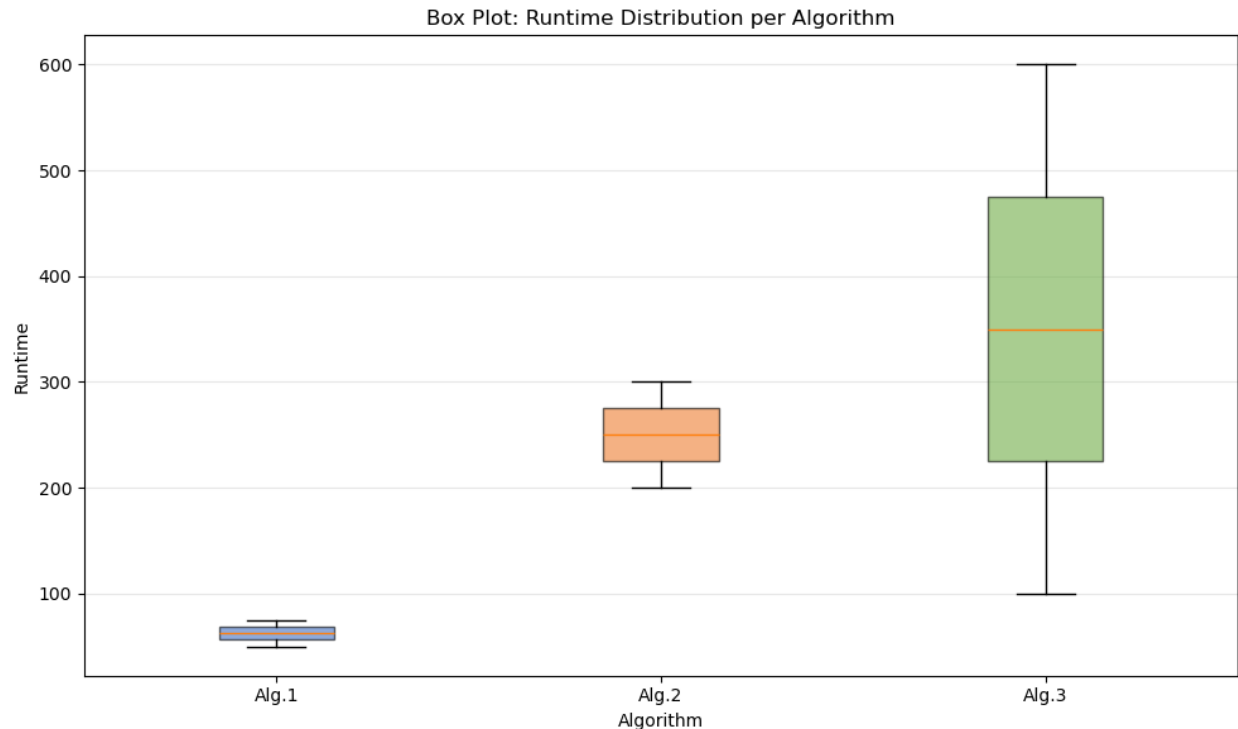
شکل 4

- **نمودار میله‌ای:** الگوریتم ۱ با بازه زمانی ۵۰ تا ۷۵، کمترین زمان اجرا را به خود اختصاص داده است. الگوریتم ۲ زمان متوسطی (بین ۲۰۰ تا ۳۰۰) و الگوریتم ۳ بیشترین زمان (۱۰۰ تا ۶۰۰) را نشان می‌دهد. فاصله عملکردی بین الگوریتم‌ها با افزایش اندازه داده‌ها بیشتر می‌شود.



شکل 5

- **نمودار خطی:** هر سه الگوریتم دارای رشد تقریباً خطی هستند. الگوریتم ۳ دارای تندترین شیب (حدود ۱ واحد به ازای هر کیلوبایت)، الگوریتم ۲ دارای شیب متوسط (۰.۲ واحد /KB) و الگوریتم ۱ ملایم‌ترین شیب (۰.۰۵ واحد /KB) است.



شکل 6

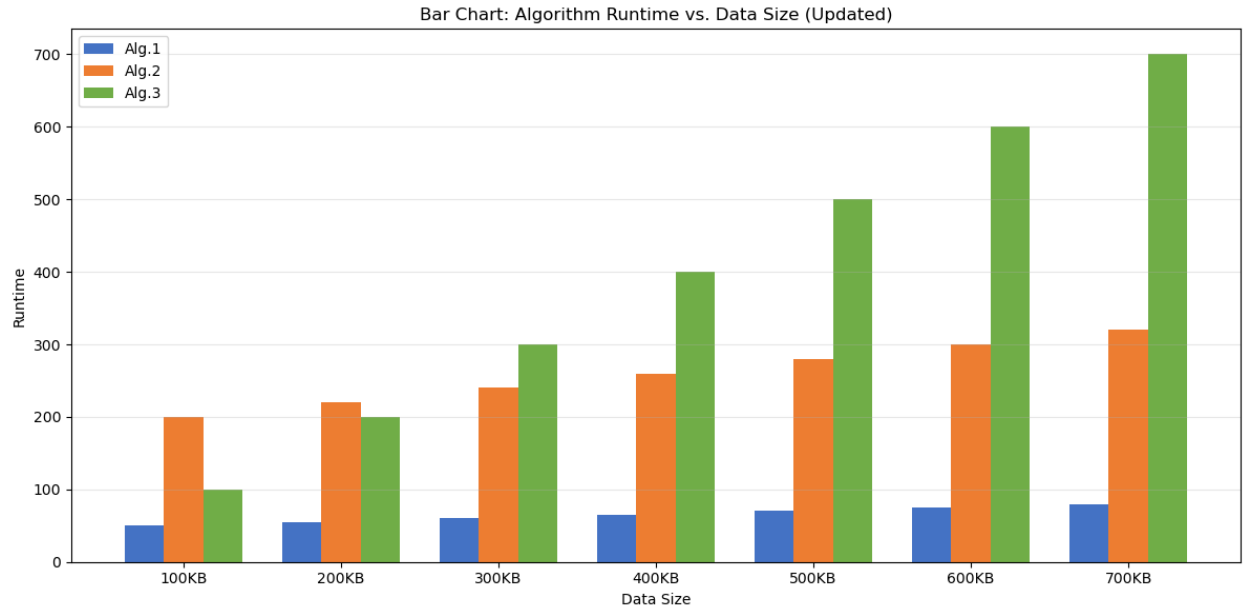
- **نمودار جعبه‌ای:** الگوریتم ۱ دارای میانه حدود ۶۲.۵ و دامنه‌ای بسیار محدود است. الگوریتم ۲ میانه حدود ۲۵۰ را ثبت کرده و الگوریتم ۳ دارای میانه ۳۵۰ با دامنه‌ای وسیع‌تر (۱۰۰ تا ۶۰۰) است. رتبه‌بندی کلی زمان اجرا به شکل $Alg.1 < Alg.2 < Alg.3$ ارزیابی می‌شود.

۵-۲ افزودن داده جدید (۷۰۰KB)

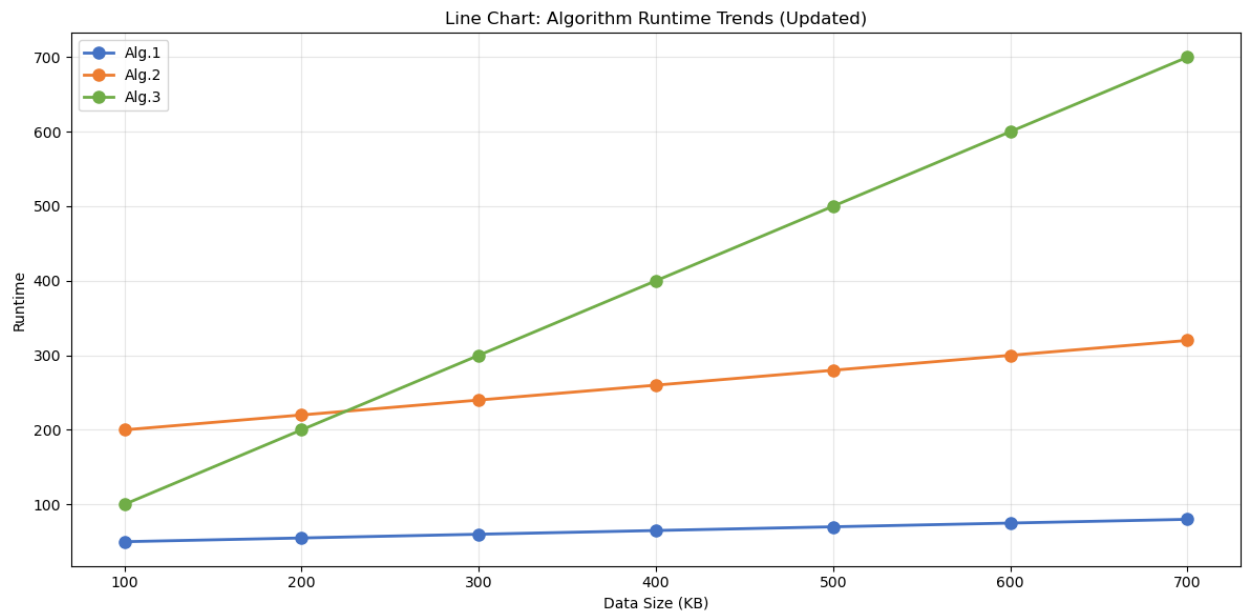
یک ردیف داده جدید مربوط به سائز ۷۰۰KB به صورت برنامه‌نویسی ساخته و به مجموعه داده‌ها اضافه شد:

```
new_row = {
    size_col: '700KB',
    'Alg.1': 80,
    'Alg.2': 320,
    'Alg.3': 700,
}
df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
```

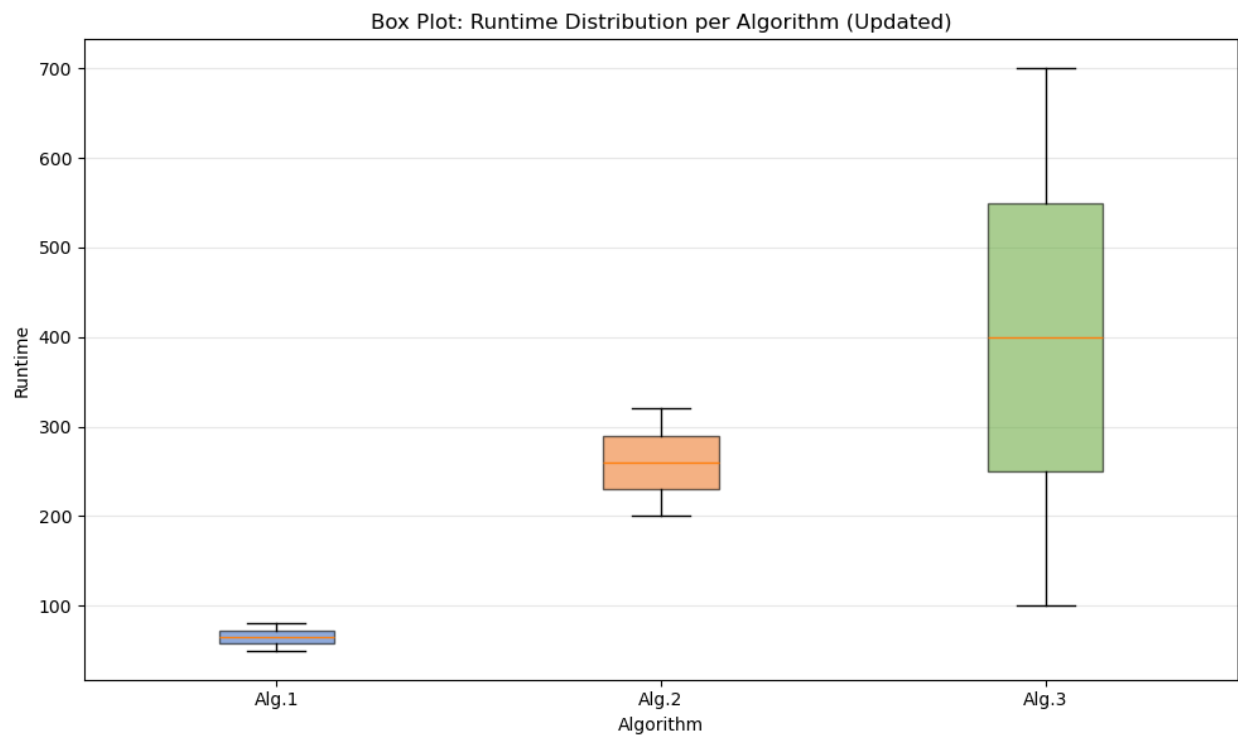
با اجرای مجدد کد رسم نمودار، نمودارها به‌روزرسانی شدند. همانگونه که در شکل ۷ و شکل ۸ و شکل ۹ مشخص است، نشان داده میشود که روند خطی برای هر سه الگوریتم تداوم یافته و دامنه پراکندگی Alg.3 به بازه ۱۰۰ تا ۷۰۰ گسترش پیدا کرده است.



شکل 7



شکل 8



شکل 9

۶-۲ میانگین زمان اجرای الگوریتم ۲

برای محاسبه میانگین الگوریتم ۲ در بازه ۱۰۰ تا ۶۰۰ کیلوبایت، قطعه کد زیر اجرا شد:

```
mask = (df_original['Size_KB'] >= 100) & (df_original['Size_KB'] <= 600)
mean_alg2 = df_original[mask]['Alg.2'].mean()
```

میانگین محاسبه شده برای این الگوریتم برابر با ۲۵۰.۰۰ است.
و خروجی کد در شکل ۱۰ آمده است.

Algorithm 2 column: 'Alg.2'

Data used (100 KB - 600 KB):

Run time for different data size		Alg.2
0	100KB	200
1	200KB	220
2	300KB	240
3	400KB	260
4	500KB	280
5	600KB	300

Mean runtime of Algorithm 2 (100-600 KB): 250.00

شکل ۱۰

سوال ۳: انتگرال گیری عددی تابع $f(x) = e^{x^2}$

۱-۳ روش پیاده سازی

مقدار انتگرال تابع در بازه $(0, 1)$ با سه رویکرد عددی شامل قاعده دوزنقه ها، قاعده سیمپسون و روش گوس-لژاندر محاسبه گردید.

۲-۳ کد پایتون

```
def f(x):  
    return np.exp(np.power(x, 2))  
  
def trapezoidal(f, a, b, n):  
    x = np.linspace(a, b, n + 1)  
    y = f(x)  
    h = (b - a) / n  
    return h/2 * (y[0] + 2*np.sum(y[1:-1]) + y[-1])  
  
def simpson(f, a, b, n):  
    x = np.linspace(a, b, n + 1)  
    y = f(x)  
    h = (b - a) / n  
    return h/3 * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-1:2]))  
  
def gauss_legendre(f, a, b, n):  
    nodes, weights = leggauss(n)  
    x_mapped = 0.5*(b-a)*nodes + 0.5*(a+b)  
    return 0.5*(b-a)*np.sum(weights * f(x_mapped))
```

۳-۳ نتایج اولیه ($n=1000$)

- قاعده دوزنقه: مقدار تخمینی $I \approx 1.462652198954$
- قاعده سیمپسون: مقدار تخمینی $I \approx 1.462651745907$

۴-۳ جدول همگرایی

با فرض مقدار مرجع $I = 1.462651745907$ ، نتایج و خطای دو روش نخست به این شرح است:

Reference (adaptive quadrature): $I = 1.462651745907$ (error estimate: $1.62e-14$)

n	Trapezoidal	Error	Simpson	Error
10	1.4671746927	$4.52e-03$	1.4626814001	$2.97e-05$
20	1.4637838918	$1.13e-03$	1.4626536249	$1.88e-06$
50	1.4628329526	$1.81e-04$	1.4626517942	$4.83e-08$
100	1.4626970498	$4.53e-05$	1.4626517489	$3.02e-09$
200	1.4626630720	$1.13e-05$	1.4626517461	$1.89e-10$
500	1.4626535581	$1.81e-06$	1.4626517459	$4.83e-12$
1000	1.4626521990	$4.53e-07$	1.4626517459	$3.02e-13$

شکل 11

۵-۳ نتایج انتگرال گیری گوس-لژاندر

- $n = 5 \text{ nodes: } I \approx 1.462651668019$ |مطلق خطای| $= 7.79e - 08$
- $n = 10 \text{ nodes: } I \approx 1.462651745907$ |مطلق خطای| $= 2.22e - 16$
- $n = 20 \text{ nodes: } I \approx 1.462651745907$ |مطلق خطای| $= 6.66e - 16$
- $n = 50 \text{ nodes: } I \approx 1.462651745907$ |مطلق خطای| $= 1.11e - 15$

۶-۳ مقایسه و تحلیل نهایی روش‌ها

مقدار مرجع انتگرال برابر با 1.4626517459 فرض شده است.

- **قاعده دوزنقه:** روشی با دقت مرتبه اول و خطای سراسری $O(h^2)$ است. با مقاطع $n = 1000$ خطایی در حدود 10^{-7} از خود نشان می‌دهد.
- **قاعده سیمپسون:** روشی با دقت مرتبه سوم و خطای سراسری $O(h^4)$ است که به طور قابل توجهی سریع‌تر همگرا می‌شود. این روش در $n = 1000$ با رسیدن به سطح $\sim 10^{-13}$ به دقت ماشین دست می‌یابد.
- **انتگرال گوسی:** تنها با 50 گره به دقت ماشین می‌رسد و به عنوان کارآمدترین روش شناخته می‌شود.
- **نتیجه‌گیری پایانی:** با توجه به همواری تابع، هر سه روش در نهایت به یک مقدار واحد همگرا شدند. در این بررسی، روش گوس-لژاندر به عنوان کارآمدترین روش، سیمپسون به عنوان بهترین فرمول از گروه نیوتون-کوتس مرکب، و قاعده دوزنقه به عنوان ساده‌ترین و در عین حال کم‌دقت‌ترین روش ارزیابی شدند.

جمع‌بندی کلی

در طول این پروژه سه هدف اصلی پیاده‌سازی و ارزیابی شد:

1. انجام درونیایی با لاگرانژ و اسپلین و استخراج دقیق ضابطه‌ها.
 2. تحلیل کامل داده‌های زمان اجرای الگوریتم‌ها با استفاده از نمودارهای آماری و گسترش مدل با داده‌های جدید.
 3. به‌کارگیری و مقایسه متدهای مختلف انتگرال‌گیری عددی شامل دوزنقه، سیمپسون و گوس-لژاندر.
- لازم به ذکر است که تمامی کدهای توسعه‌یافته برای این اهداف به ترتیب در فایل‌های s01.ipynb، s02.ipynb و s03.ipynb ثبت و ذخیره شده‌اند.